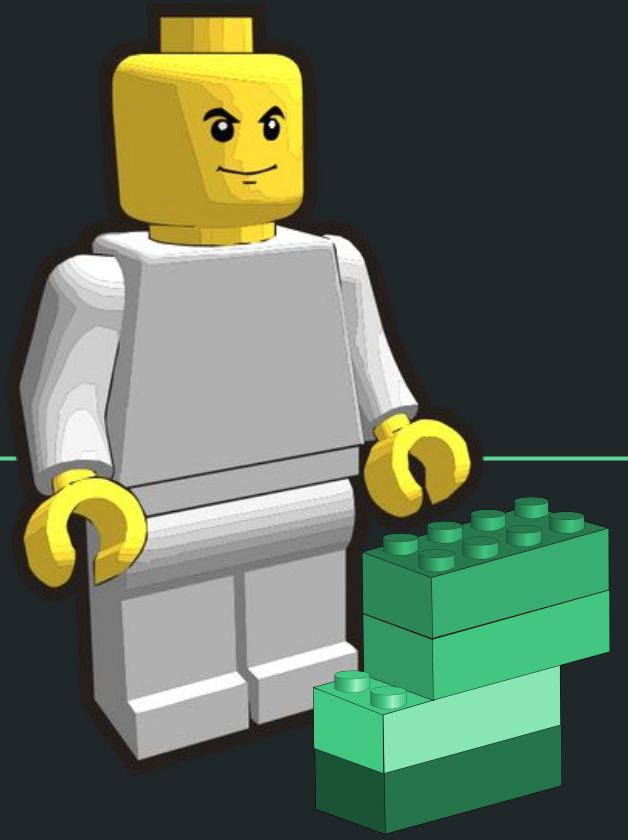
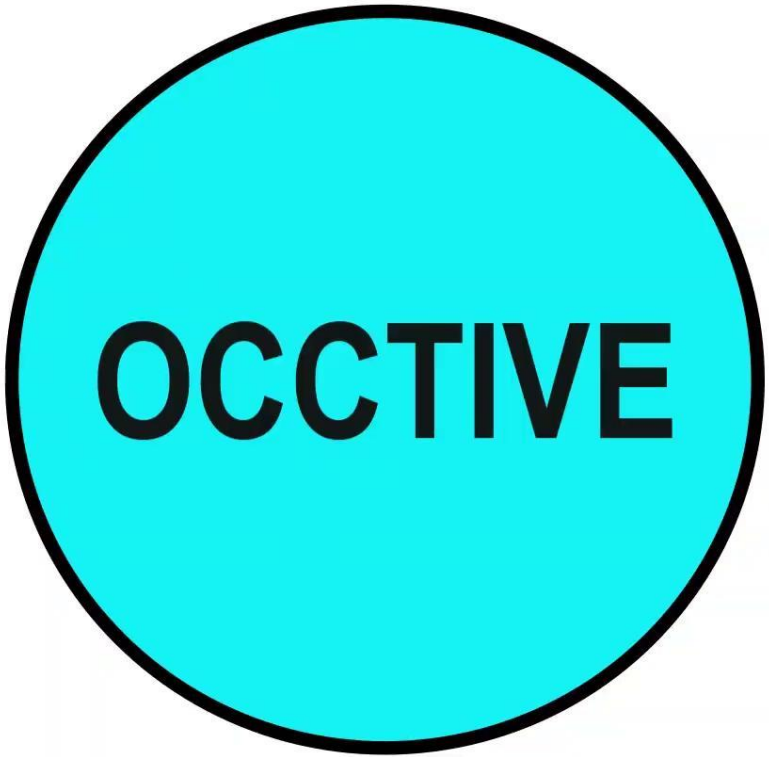
Algorithmic thinking & Modular design




OCCTIVE

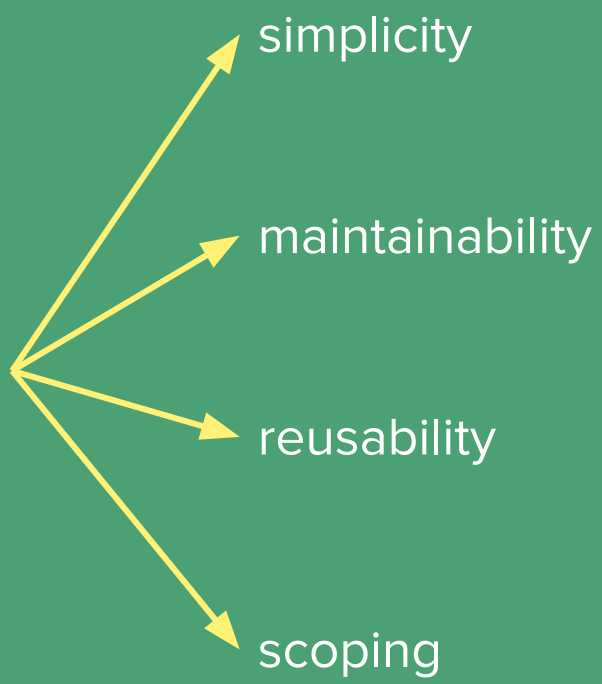
Python supports **modular programming** in multiple ways.

Functions and **classes** are examples of tools for low-level modular programming.

Python **modules** are a higher-level modular programming construct, where we can collect related variables, functions and classes in a module.

Modules are often bundled up into **packages**.

Modular design, like modular programming, is the approach of designing and building things as independent modules.



More information: https://realpython.com/lessons/python-modules-packages-overview/

Importing **modules**

- Every file that has the file extension .py & contains Python code can be seen or is a module (there is no special syntax required to make such a file a module)
- We can import these using `import module` (e.g., `import urllib`)
  - We also have the option to nickname modules when we import
- If only certain objects of a module are needed, we can import only those: `from Bio import Entrez`
- Instead of explicitly importing certain objects from a module, it's also possible to import everything by using an asterisk in the import: `from Bio import *`
- If you change a module and if you want to reload it, you must restart the kernel (interpreter).

Content adapted from: https://realpython.com/lessons/scripts-modules-packages-and-libraries/

Executing modules as **scripts**

- A **script** is a Python file that's intended to be run directly. When you run it, it should do something. This means that scripts will often contain code written *outside* the scope of any classes or functions.
- A Python module is essentially a script, and can be run as one.
- In Jupyter Notebooks, we can run this as a magic command:
  ```
  %run hello.py
  ```

Content adapted from: https://realpython.com/lessons/scripts-modules-packages-and-libraries/

Project Organization

- **Notebooks**: good for interactive development
  - For when seeing the inputs and outputs of code running needs to be seen start to finish
  - file ends in .ipynb
- **Modules**: for storing mature Python code, that you can import
  - you don't *use* the functions in there, just define them
  - file ends in .py
- **Scripts**: a Python file for executing a particular task
  - this takes an input and does something start to finish
  - file ends in .py

Let’s illustrate how this might work, using our signal processing notebook.